STA 243: Homework 1

- Homework due in Canvas: 04/22/2025 at 9:00 PM.

1. **[10 points]** The purpose of this question is to brush-up your linear algebra background. Let $\boldsymbol{A}$ be an $n \times n$ square matrix. Show that the following statements are equivalent:

   (a) The columns of $\boldsymbol{A}$ are orthonormal vectors.

   (b) $\boldsymbol{A}\boldsymbol{A}^\top = \boldsymbol{A}^\top \boldsymbol{A} = \boldsymbol{I}$, where $\boldsymbol{I}$ is the identity matrix.

   (c) The rows of $\boldsymbol{A}$ are orthonormal vectors.

Proof: Let $\mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_n$ be the columns of A. And let the i-th row of A be $\mathbf{r}_i^\top$. Then the entries of $A^\top A$ and $AA^\top$ are

$$(A^\top A)_{ij} = \mathbf{a}_i^\top \mathbf{a}_j, \qquad (AA^\top)_{ij} = \mathbf{r}_i^\top \mathbf{r}_j.$$

Further, define δ_{ij} to be defined as follows:

$$\delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

(a) $\Rightarrow$ (b) If the columns $\{\mathbf{a}_i\}$ are orthonormal, then for all i, j

$$\mathbf{a}_i^\top \mathbf{a}_j = \delta_{ij} = 0 \ \& \ \mathbf{a}_i^\top \mathbf{a}_i = \delta_{ii} = 1 \Longrightarrow A^\top A = I.$$

Since $A^\top A = I$ and A is square, A is invertible with $A^{-1} = A^\top$, and so

$$AA^\top = I.$$

Hence $AA^\top = A^\top A = I$, proving (b).

(b) $\Rightarrow$ (c) If $AA^\top = I$, then for all $i \neq j$,

$$\mathbf{r}_i^\top \mathbf{r}_j = (AA^\top)_{ij} = \delta_{ij} = 0,$$

so the rows $\{\mathbf{r}_i\}$ are orthonormal, proving (c).

(c) $\Rightarrow$ (a) If the rows $\{\mathbf{r}_i\}$ are orthonormal, then

$$AA^\top = I.$$

Again A is square and invertible, hence $A^{-1} = A^\top$, which implies

$$A^\top A = I,$$

so

$$\mathbf{a}_i^\top \mathbf{a}_j = (A^\top A)_{ij} = \delta_{ij} = 0 \ \& \ \mathbf{a}_i^\top \mathbf{a}_i = \delta_{ii} = 1,$$

i.e. the columns $\{\mathbf{a}_i\}$ are orthonormal, proving (a).

Since $(a) \Rightarrow (b) \Rightarrow (c) \Rightarrow (a)$, all three statements are equivalent. $\square$

2. **[10 points]** Show how do you go from Equation (2) to Equation (3) in the variance calculation in slide 20 of the RandLA.pdf slides.

Proof: Write

$$X_\ell \;=\; \sum_{k=1}^{n} \frac{1}{r\,p_k}\, A_{:,k}\, B_{k,:}\, \mathbf{1}\{i_\ell = k\} \quad \text{so that} \quad M \;=\; \sum_{\ell=1}^{r} X_\ell.$$

Since $\mathbf{1}\{i_\ell = k\}^2 = \mathbf{1}\{i_\ell = k\}$ and $\Pr(i_\ell = k) = \mathbb{E}\left(\mathbf{1}\{i_\ell = k\}\right) = p_k$, we get

$$\mathbb{E}[X_\ell] = \sum_{k=1}^{n} \frac{1}{r\,p_k}\, A_{:,k}\, B_{k,:}\, \mathbb{E}[\mathbf{1}\{i_\ell = k\}] = \sum_{k=1}^{n} \frac{p_k}{r\,p_k}\, A_{:,k}\, B_{k,:} = \frac{1}{r} \sum_{k=1}^{n} A_{:,k}\, B_{k,:},$$

$$\begin{aligned}
\mathbb{E}[X_\ell^2] &= \mathbb{E}\Big[\sum_{k=1}^{n} \frac{1}{r\,p_k}\, A_{:,k}\, B_{k,:}\, \mathbf{1}\{i_\ell = k\}\Big]^2 = \sum_{k=1}^{n} \frac{1}{r^2 p_k^2}\, A_{:,k}^2\, B_{k,:}^2\, \mathbb{E}[\mathbf{1}\{i_\ell = k\}] \\
&= \sum_{k=1}^{n} \frac{p_k}{r^2 p_k^2}\, A_{:,k}^2\, B_{k,:}^2 = \sum_{k=1}^{n} \frac{1}{r^2 p_k}\, A_{:,k}^2\, B_{k,:}^2.
\end{aligned}$$

Hence

$$\mathrm{Var}[X_\ell] = \mathbb{E}[X_\ell^2] - \big(\mathbb{E}[X_\ell]\big)^2 = \sum_{k=1}^{n} \frac{1}{r^2 p_k}\, A_{:,k}^2\, B_{k,:}^2 - \frac{1}{r^2}\Big(\sum_{k=1}^{n} A_{:,k}\, B_{k,:}\Big)^2.$$

Summing over all entries (i,j) and all $\ell = 1, \ldots, r$,

$$\begin{aligned}
\sum_{i,j} \mathrm{Var}[M_{ij}] &= \sum_{\ell=1}^{r} \sum_{i,j} \mathrm{Var}[X_{\ell,ij}] = r \sum_{i,j} \mathrm{Var}[X_{1,ij}] \\
&= \sum_{i,j} \left(\sum_{k=1}^{n} \frac{1}{r\,p_k} A_{ik}^2 B_{kj}^2 - \frac{1}{r} (AB)_{ij}^2 \right) \\
&= \frac{1}{r} \sum_{k=1}^{n} \frac{\sum_{i,j} A_{ik}^2 B_{kj}^2}{p_k} - \frac{1}{r} \sum_{i,j} (AB)_{ij}^2.
\end{aligned}$$

Rewrite the second term using the Frobenius norm,

$$\sum_{i,j} \mathrm{Var}[M_{ij}] = \frac{1}{r} \sum_{k=1}^{n} \frac{\|A_{:,k}\|_2^2\, \|B_{k,:}\|_2^2}{p_k} - \frac{1}{r} \|AB\|_F^2.$$

$\square$

3. **[10 points] Randomized matrix multiplication**:

   (a) Implement the algorithm presented in class for randomized matrix multiplication.

   (b) Apply the algorithm to the provided matrices, selecting a number of columns $r = 20, 50, 100, 200$. The matrices for this problem can be found in the attached files "STA243_homework_1_matrix_A.csv" and "STA243_homework_1_matrix_B.csv".

   (c) Calculate the relative approximation error $||M - AB||_F/(||A||_F||B||_F)$ for each of the estimates found in (b). Provide your results in a table.

   (d) Visualize the estimates from (b) in Python (you can use Matplotlib).

Solution: The algorithm for randomized matrix multiplication is implemented in the file `AZ_HW1.ipynb`. The relative approximation error is calculated using the following formula:

$$\frac{\|M - AB\|_F}{\|A\|_F\|B\|_F}$$

And the results are presented in the table below:

r	relative approximation error
20	0.190787
50	0.128489
100	0.095043
200	0.074441

When the number of columns r increases, the relative approximation error decreases. This is because more columns provide a better approximation of the original matrix product. And the results are as expected, as the relative approximation error decreases with increasing r. And the visualization of the estimates from (b) is shown in the figure below:

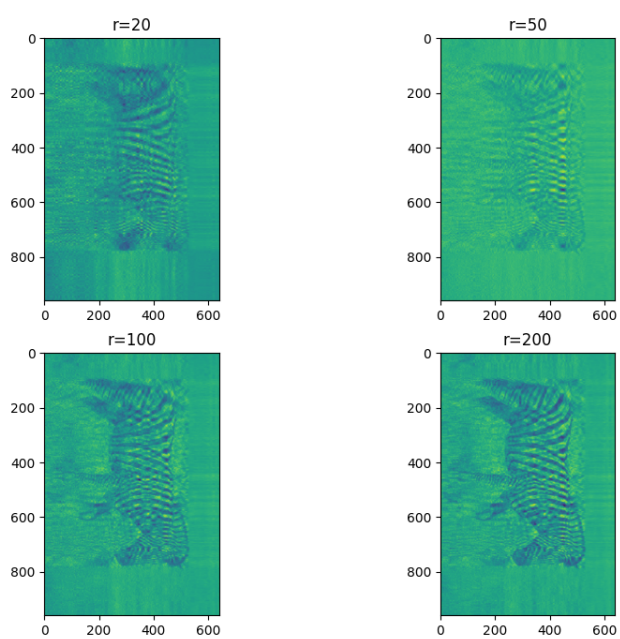

Figure 1: Visualization of the estimates from (b)

And the original matrices are shown in the figure below:

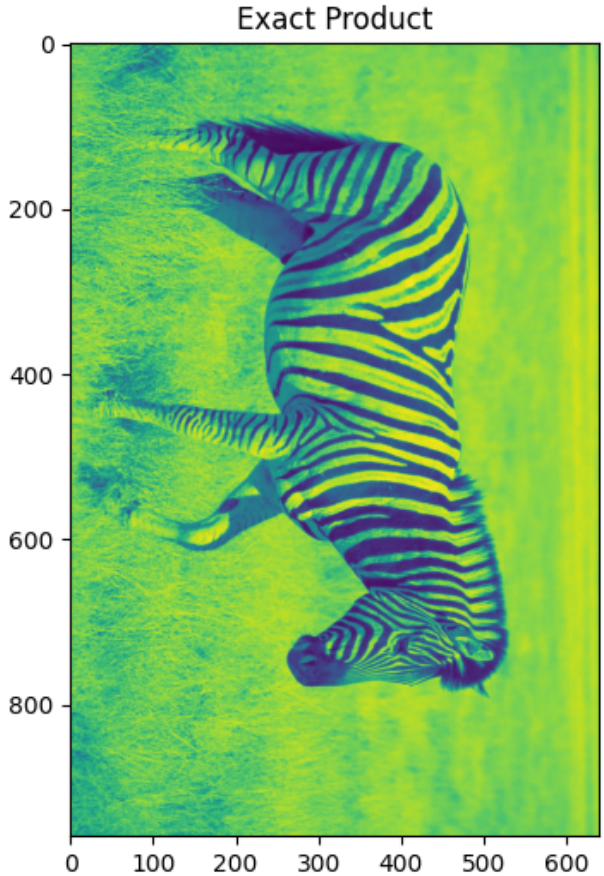

Figure 2: Original matrices

From the figures, we can see that the matrix estimates [1] are close to the original matrices [2] when r is larage, and the relative approximation error is small. This indicates that the randomized matrix multiplication algorithm is effective in approximating the product of two matrices.

4. **[10 points] Power method**: Let $\mathbf{X}$ be a 10×10 matrix such that

$$\mathbf{X} = \lambda(\boldsymbol{v}\boldsymbol{v}^\top) + \mathbf{E},$$

where $\mathbf{E}$ is a *random matrix* with each entry being an i.i.d. standard Gaussian variable. Note that $\mathbf{X}$ is a rank-1 matrix perturbed with a random noise matrix ($\mathbf{E}$). Your goal in this problem is to *estimate* the eigenvector $\boldsymbol{v}$. The file `power_sim.py` consists of a test routine that fixes the true eigenvector, initial vector used for power method and the noise matrix. The end goal is produce a plot of λ versus how well the estimated eigenvector (using the power method) is correlated (measured via inner-product) with the true eigenvector. For this you have to implement your own power method (function `power_iteration`). Complete the code and run the test routine to produce the plot.

Solution: The power method is implemented in the file `AZ_HW1.ipynb`. The function `power_iteration` takes the matrix $\mathbf{X}$, the initial vector $\mathbf{v}_0$, and the number of iterations as input and returns the estimated eigenvector. The test routine generates a plot of λ versus the inner product between the estimated eigenvector and the true eigenvector. The plot is shown below:

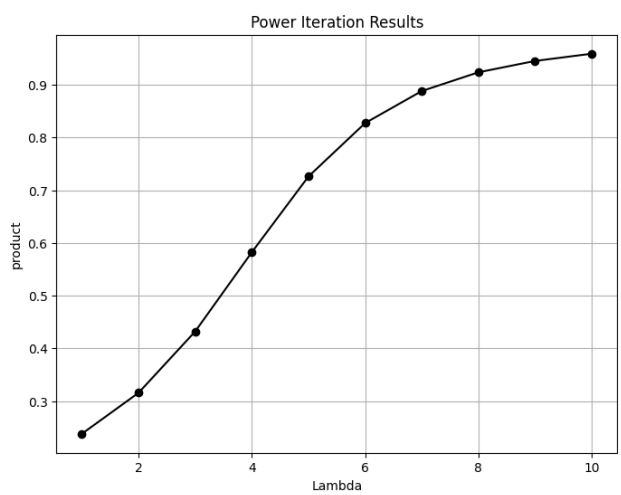

Figure 3: Plot of λ versus inner product

From the plot, we can see that as λ increases, the inner product between the estimated eigenvector and the true eigenvector also increases. This indicates that the power method is effective in estimating the eigenvector of the matrix $\mathbf{X}$.

5. **[10 points] Sketching for Least-squares**:

   (a) Implement the Sketched-OLS algorithm presented in class.

   (b) Generate a 1048576×20 design matrix $\mathbf{X}$ and a 1048576×1 response $\mathbf{y}$ with elements drawn iid from a Uniform$(0, 1)$ distribution.

   (c) Compare the calculation time for the full least squares problem and the sketched OLS. For the matrix $\mathbf{\Phi} = \mathbf{S}^T\mathbf{HD}$, first calculate $\mathbf{X}_* = \mathbf{\Phi X}$ and $\mathbf{y}_* = \mathbf{\Phi y}$. Once finished, import time package in python to time the calculation of $(\mathbf{X}_*^T\mathbf{X}_*)^{-1}\mathbf{X}_*^T\mathbf{y}_*$ and compare to the calculation time of $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$. Repeat these steps for $\epsilon = .1, .05, .01, .001$ and present your results in a table.

Solution: The Sketched-OLS algorithm is implemented in the file `AZ_HW1.ipynb`. The design matrix $\mathbf{X}$ and response $\mathbf{y}$ are generated using the Uniform$(0, 1)$ distribution. The calculation time for the full least squares problem and the sketched OLS is compared using the time package in Python. The results are presented in the table below:

ϵ	Computation Time
0.1	0.000253916
0.05	0.000325203
0.01	0.00128531
0.001	0.00987411
OLS	0.0443492

From the table, we can see that as ϵ decreases, the computation time for the sketched OLS increases. However, the sketched OLS is still significantly faster than the full least squares problem. This indicates that the sketched OLS algorithm is effective in reducing the computation time for large-scale least squares problems.

Pledge:

Please sign below (print full name) after checking (✓) the following. If you can not honestly check each of these responses, please email me at kbala@ucdavis.edu to explain your situation.

- We pledge that we are honest students with academic integrity and we have not cheated on this homework.
- These answers are our own work.
- We did not give any other students assistance on this homework.
- We understand that to submit work that is not our own and pretend that it is our is a violation of the UC Davis code of conduct and will be reported to Student Judicial Affairs.
- We understand that suspected misconduct on this homework will be reported to the Office of Student Support and Judicial Affairs and, if established, will result in disciplinary sanctions up through Dismissal from the University and a grade penalty up to a grade of "F" for the course.

Name: Adrian Zhou